

```
import os
```

```
import os
os.environ["GROQ_API_KEY"] = "gsk_zsms9GdU4p3iIDRHoGFEGWdyb3FY6qGVEBQTr3k3HHDOvuJTy8R" # Replace with your actual Groq API
```

```
!pip install groq
```

```
Collecting groq
  Downloading groq-1.5.0-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from groq) (4.13.0)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from groq) (1.9.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from groq) (0.28.1)
Requirement already satisfied: pydantic<3,>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from groq) (2.12.3)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from groq) (1.3.1)
Requirement already satisfied: typing-extensions<5,>=4.14 in /usr/local/lib/python3.12/dist-packages (from groq) (4.15.0)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.5.0->groq) (3.18)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq) (2026.5.20)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->groq) (0.14.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (2.41.4)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,>=1.9.0->groq) (0.4.2)
Downloading groq-1.5.0-py3-none-any.whl (143 kB)
143.7/143.7 kB 5.1 MB/s eta 0:00:00

Installing collected packages: groq
Successfully installed groq-1.5.0
```

```
import os
from groq import Groq

# Make sure to set your GROQ_API_KEY environment variable in a previous cell
# For example: os.environ["GROQ_API_KEY"] = "YOUR_ACTUAL_API_KEY"
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
completion = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {
            "role": "user",
            "content": "Hello, how are you?"
        }
    ],
    temperature=1,
    max_completion_tokens=1024,
    top_p=1,
    stream=True,
    stop=None
)

for chunk in completion:
    print(chunk.choices[0].delta.content or "", end="")
```

I'm functionally well, thank you for asking. I'm here to help answer any questions you might have or discuss topics you're interested in.

```
import os
from groq import Groq

client = Groq()
# Removed os.path.dirname(__file__) as it's not defined in Colab notebooks
# Ensure this path correctly points to your file within Google Drive
filename = "/content/drive/MyDrive/freesound_community-broken-audio-file-distortion-104395.mp3"

# Ensure the file exists and is accessible
if not os.path.exists(filename):
    raise FileNotFoundError(f"Audio file not found at: {filename}. Please check the path and ensure your Google Drive is mounted.")

with open(filename, "rb") as file:
    transcription = client.audio.transcriptions.create(
        file=(os.path.basename(filename), file.read()),
        model="whisper-large-v3",
        temperature=0,
        response_format="verbose_json",
    )
    print(transcription.text)
```

Vroom vroom vroom vroom vroom

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

```
from groq import Groq

client = Groq()
completion = client.chat.completions.create(
    model="qwen/qwen3-32b",
    messages=[
        {
            "role": "user",
            "content": ""
        }
    ],
    temperature=0.6,
    max_completion_tokens=4096,
    top_p=0.95,
    reasoning_effort="default",
    stream=True,
    stop=None
)

for chunk in completion:
    print(chunk.choices[0].delta.content or "", end="")
```

<think>

Okay, the user just sent a message with a series of "a" letters. Let me figure out what they need.

First, I notice they wrote "a" multiple times. Maybe they're testing if I'm paying attention or trying to see how I'll respond.

I should acknowledge their message and ask for clarification. They might need help with something specific but aren't sure how to ask.

I should keep the response friendly and open-ended to encourage them to provide more details. Let me make sure I'm not missing anything.

Also, maybe they want to know if I can handle non-verbal cues or patterns. I should stay within the guidelines and not make assumptions.

Alright, the best approach is to respond with a simple message asking them to clarify their request.

</think>

It seems like you might be testing or have a typo. How can I assist you today? Feel free to ask any question or let me know if you need more help.

Start coding or [generate](#) with AI.